Receta AWS

Lanzamiento instancia EC2 con servicios Gunicorn y Nginx para DRF

ETAPA 1: Preparar la infraestructura en AWS (EC2 y VPC)

1. Crear una instancia EC2 (capa gratuita o free tier)

* Entra a tu consola AWS → **EC2** → "Lanzar instancia".
* Nombre: `django-todolist (nombre del proyecto)`
* **AMI**: Ubuntu Server 22.04 LTS (x64)
* **Tipo de instancia**: `t2.micro o t3.micro` (capa gratuita)
* **Par de claves (Key Pair)**: crea o usa una existente (`claves.pem`)
* **Configuración de red (VPC y subred)**:
    - Usa la **VPC por defecto** (no es necesario crear una nueva).
    - Habilita la **asignación automática de IP pública**.
* **Grupo de seguridad (Security Group)**:
    - Permitir tráfico entrante:
        - SSH (puerto 22) — desde tu IP.
        - HTTP (puerto 80) — desde cualquier IP.
        - Opcional: HTTPS (443) si planeas agregar SSL más adelante.
        - Activar el protocolo ICMP (ping) para pruebas iniciales de conexión.

Elastic IP: Una vez creada la instancia, asígnale una IP elástica (EIP) para que no cambie al reiniciar:

* EC2 → Red y seguridad → "IP elásticas" → Asignar nueva IP → Asociar con tu instancia.

ETAPA 2: Conectarte a la instancia (desde Windows 10)

1. Descarga el archivo `claves.pem` (si no lo tienes).
2. Instala **Windows Terminal** o usa **PowerShell / CMD** con **OpenSSH**.
3. Conéctate:

```
cd C:\Users\<TU_USUARIO>\Downloads
ssh -i "claves.pem" ubuntu@<TU_IP_ELASTICA>
```

Consejo: asegúrate de que los permisos de la clave sean privados:

```
chmod 400 claves.pem
```

ETAPA 3: Configurar el servidor Ubuntu

Actualiza el sistema:

```
sudo apt update && sudo apt upgrade -y
```

Instala los paquetes necesarios:

```
sudo apt install python3-pip python3-venv nginx git -y
```

ETAPA 4: Crear el proyecto Django + DRF (ToDoList)

1. Crea un entorno virtual:

```
mkdir ~/apps && cd ~/apps
python3 -m venv venv
source venv/bin/activate
```

2. Instala Django y DRF:

```
pip install django djangorestframework gunicorn
```

3. Crea el proyecto:

```
django-admin startproject todolist_project .
```

4. Crea la app:

```
python manage.py startapp tasks
```

5. Agrega la app al `settings.py`:

```
ALLOWED_HOSTS = ['*']

INSTALLED_APPS = [
    'rest_framework',
    'tasks',
]
```

6. Define un modelo simple: `tasks/models.py`

```
from django.db import models

class Task(models.Model):
    title = models.CharField(max_length=200)
    done = models.BooleanField(default=False)

    def __str__(self):
        return self.title
```

7. Crea el serializer y la vista DRF: `tasks/serializers.py`

```
from rest_framework import serializers
from .models import Task

class TaskSerializer(serializers.ModelSerializer):
    class Meta:
        model = Task
        fields = '__all__'
```

tasks/views.py

```
from rest_framework import viewsets
from .models import Task
from .serializers import TaskSerializer

class TaskViewSet(viewsets.ModelViewSet):
    queryset = Task.objects.all()
    serializer_class = TaskSerializer
```

8. Configura las rutas: todolist_project/urls.py

```
from django.contrib import admin
from django.urls import path, include
from rest_framework import routers
from tasks.views import TaskViewSet

router = routers.DefaultRouter()
router.register(r'tasks', TaskViewSet)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include(router.urls)),
]
```

9. Migra y prueba:

```
python manage.py migrate
python manage.py runserver 0.0.0.0:8000
```

Ahora puedes visitar http://<tu-ip-elastica>:8000/api/tasks/

Con esto pruebas que funciona la instalación de DRF, aun no los servidores finales.

ETAPA 5: Configurar Gunicorn (WSGI server)

Instala gunicorn si no lo hiciste:

```
pip install gunicorn
```

Prueba que funcione:

```
gunicorn --bind 0.0.0.0:8000 todolist_project.wsgi
```

Detén con Ctrl+C.

Crea un **servicio systemd** para gestionarlo automáticamente:

```
sudo nano /etc/systemd/system/gunicorn.service
```

Pega:

```
[Unit]
Description=Gunicorn daemon for Django ToDoList
After=network.target

[Service]
User=ubuntu
Group=www-data
WorkingDirectory=/home/ubuntu/apps
ExecStart=/home/ubuntu/apps/venv/bin/gunicorn --workers 3 --bind
unix:/home/ubuntu/apps/todolist.sock todolist_project.wsgi:application
```

```
[Install]
WantedBy=multi-user.target
```

Inicia y habilita:

```
sudo systemctl start gunicorn
sudo systemctl enable gunicorn
```

En este punto GUNICORN creará el SOCKET para generar la conexión entre el cliente y el servidor.

ETAPA 6: Configurar Nginx como proxy inverso

Crea una configuración para tu sitio:

```
sudo nano /etc/nginx/sites-available/todolist
```

Contenido:

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://unix:/home/ubuntu/apps/todolist.sock;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    error_log /var/log/nginx/todolist_error.log;

    access_log /var/log/nginx/todolist_access.log;
}
```

Activa el sitio:

```
sudo ln -s /etc/nginx/sites-available/todolist /etc/nginx/sites-enabled
sudo nginx -t
sudo systemctl restart nginx
```

Aquí estamos habilitando el sitio y dejándolo listo para ser usado por el servidor.

Ver logs de Nginx:

```
sudo tail -n 20 /var/log/nginx/todolist_error.log
```

ETAPA 8: Probar y finalizar

Reiniciar todo y poner orden a nuestro despliegue.

```
sudo systemctl daemon-reload
sudo systemctl restart gunicorn
sudo systemctl restart nginx
sudo systemctl status gunicorn
sudo systemctl status nginx
```

Y luego prueba de nuevo desde tu navegador:

http://<tu ip publica>/api/tasks/

Deberías ver la interfaz DRF funcionando a través de **Nginx + Gunicorn + Django REST Framework**

Soluciones a problemas posibles:

Solución posible para ETAPA 2

Mover la clave a una carpeta privada, por ejemplo:

mkdir C:\AWS_KEYS

copy "<Origen>\claves.pem" C:\AWS_KEYS\

Y luego:

icacls "C:\AWS_KEYS\claves.pem" /inheritance:r

icacls "C:\AWS_KEYS\claves.pem" /grant:r "%username%:R"

ejemplo: icacls "C:\AWS_KEYS\claves.pem" /grant:r "jose:R"

Así la clave queda protegida en un directorio controlado.

Solución posible para ETAPAS 5-6

Error encontrado	Causa más probable	Solución
502 Bad Gateway	Socket no creado	Revisar gunicorn.service y reiniciar
(13) Permission denied	Permisos del socket	chown ubuntu:www-data y chmod 660
(2) No such file	Ruta incorrecta del socket	Verificar proxy_pass en Nginx
(111) Connection refused	Gunicorn no en ejecución	systemctl restart gunicorn
Gunicorn falla al iniciar	Ruta de venv o wsgi incorrecta	Corregir archivo .service

Verifica permisos del árbol de directorios

Ejecuta:

```
namei -l /home/ubuntu/apps/todolist.sock
Verás algo como:
drwx------ ubuntu ubuntu /home
drwxr-x--- ubuntu ubuntu ubuntu
srw-rw---- ubuntu www-data todolist.sock
Si /home/ubuntu aparece con drwx------
```

Entonces **www-data no puede ni entrar al directorio**, por eso da *permission denied*.
Hay dos formas de resolverlo correctamente:

Solución A (recomendada): permitir acceso al grupo www-data

a) Añadir ubuntu y www-data al mismo grupo

```
sudo usermod -aG www-data ubuntu
sudo usermod -aG ubuntu www-data
```

b) Ajustar permisos de los directorios

```
sudo chmod 750 /home/ubuntu
sudo chmod 770 /home/ubuntu/apps
```

Esto da:

- Lectura y ejecución a ubuntu
- Acceso controlado a www-data (el grupo que usa Nginx)

c) Ajustar propietario del socket

```
sudo chown ubuntu:www-data /home/ubuntu/apps/todolist.sock
sudo chmod 660 /home/ubuntu/apps/todolist.sock
```

d) Reiniciar servicios

```
sudo systemctl restart gunicorn
sudo systemctl restart nginx
```

Luego prueba otra vez en tu navegador:

http://<tu ip elastica>/api/tasks/

Solución B (más directa, pero menos segura)

Si necesitas una prueba rápida sin tocar grupos:

```
sudo chmod 755 /home/ubuntu
sudo chmod 775 /home/ubuntu/apps
sudo chown ubuntu:www-data /home/ubuntu/apps/todolist.sock
sudo chmod 666 /home/ubuntu/apps/todolist.sock
```

Esto abrirá permisos de lectura/escritura temporalmente.
Verifica que funciona, y luego vuelve a restringirlos (para producción no se recomienda 666).

Validación posterior

Verifica nuevamente el socket:

```
ls -ld /home /home/ubuntu /home/ubuntu/apps /home/ubuntu/apps/todolist.sock
```

Resultado esperado:

```
drwxr-xr-x root root        /home
drwxr-x--- ubuntu ubuntu    /home/ubuntu
drwxrwx--- ubuntu www-data  /home/ubuntu/apps
srw-rw---- ubuntu www-data  /home/ubuntu/apps/todolist.sock
```

Ahora Nginx (que corre como www-data) podrá “atravesar” /home/ubuntu/ hasta el socket.

Confirmar con prueba de conexión manual

Ejecuta nuevamente:

```
curl --unix-socket /home/ubuntu/apps/todolist.sock http://localhost/
```

Si responde con JSON o HTML → Gunicorn y permisos correctos.
Luego prueba vía Nginx:

```
curl -I http://localhost/
```

Debe mostrar:

HTTP/1.1 200 OK

Server: nginx/1.18.0 (Ubuntu)

Y finalmente abre en tu navegador:

http://<tu ip publica>/api/tasks/

Verificación final del flujo

Componente	Usuario	Permisos esperados
Gunicorn	ubuntu	crea el socket
Nginx	www-data	debe poder leer el socket
Socket	ubuntu:www-data	rw-rw----
Carpeta /home/ubuntu/apps	ubuntu:www-data	rwxrwx---

Configurar STATIC_ROOT en settings.py para ver las graficas de /ADMIN y /API

Edita el archivo:

```
sudo nano /home/ubuntu/apps/todolist_project/settings.py
```

Agrega al final:

```
STATIC_URL = '/static/'
STATIC_ROOT = '/home/ubuntu/apps/static/'
```

Guarda y cierra.

Recopilar archivos estáticos

Activa el entorno virtual y ejecuta:

```
cd /home/ubuntu/apps
source venv/bin/activate
python manage.py collectstatic
```

Cuando pregunte:

```
You have requested to collect static files at the destination location
/home/ubuntu/apps/static/
```

responde “yes”.

Esto copiará todos los archivos CSS/JS de Django y DRF dentro de /home/ubuntu/apps/static/.

Verifica:

```
ls /home/ubuntu/apps/static/
```

Deberías ver carpetas como admin, rest_framework, etc.

Configurar Nginx para servir los estáticos

Edita tu archivo de configuración del sitio:

```
sudo nano /etc/nginx/sites-available/todolist
```

Agrega (o ajusta) este bloque **antes del location /**:

```
server {
    listen 80;
    server_name _;
    location /static/ {
        alias /home/ubuntu/apps/static/;
    }
    location / {
        include proxy_params;
        proxy_pass http://unix:/home/ubuntu/apps/todolist.sock;
    }
    error_log /var/log/nginx/todolist_error.log;
    access_log /var/log/nginx/todolist_access.log;
}
```

Guarda y valida la sintaxis:

```
sudo nginx -t
```

Si dice syntax is ok, reinicia:

```
sudo systemctl restart nginx
```

Ajustar permisos de los archivos estáticos

Asegúrate de que Nginx (usuario www-data) tenga acceso:

```
sudo chown -R ubuntu:www-data /home/ubuntu/apps/static
sudo chmod -R 755 /home/ubuntu/apps/static
```

Prueba nuevamente

Recarga en el navegador:

http://<tu ip publica>/api/

y luego:

http:// <tu ip publica>/admin/

Ahora los elementos gráficos (CSS y JS) deberían cargarse correctamente.